

Alpha-Beta Pruning and Expectimax

Exact adversarial bounds and probabilistic chance-node backups

Learning objectives

By the end of this lesson, you should be able to:

- 01 — Define alpha and beta as backed-up bounds.
- 02 — Trace exact alpha-beta prune conditions.
- 03 — Explain why pruning preserves the minimax root value.
- 04 — Relate move ordering to effective branching factor.
- 05 — Compute expectimax chance-node values.
- 06 — Choose between minimax and expectimax from environment assumptions.

Alpha-beta pruning is exact

A branch may be skipped only after bounds prove that it cannot change an ancestor's decision.

- MAX prunes when $v \geq \beta$.
- MIN prunes when $v \leq \alpha$.
- Unpruned minimax and alpha-beta return the same value.

Bound

Alpha is MAX's best guaranteed value; beta is MIN's best guaranteed value.

Alpha and beta summarize ancestor alternatives

alpha = best MAX value on path; beta = best MIN value on path; prune when $\alpha \geq \beta$

- Alpha is a lower bound for MAX.
- Beta is an upper bound for MAX under MIN control.
- Bounds tighten monotonically down a path.
- A cutoff occurs only when an ancestor already has a better choice.

ENGINEERING IMPLICATION

Initialize $\alpha = -\infty$ and $\beta = +\infty$ at the root.

Pruning changes work, not semantics

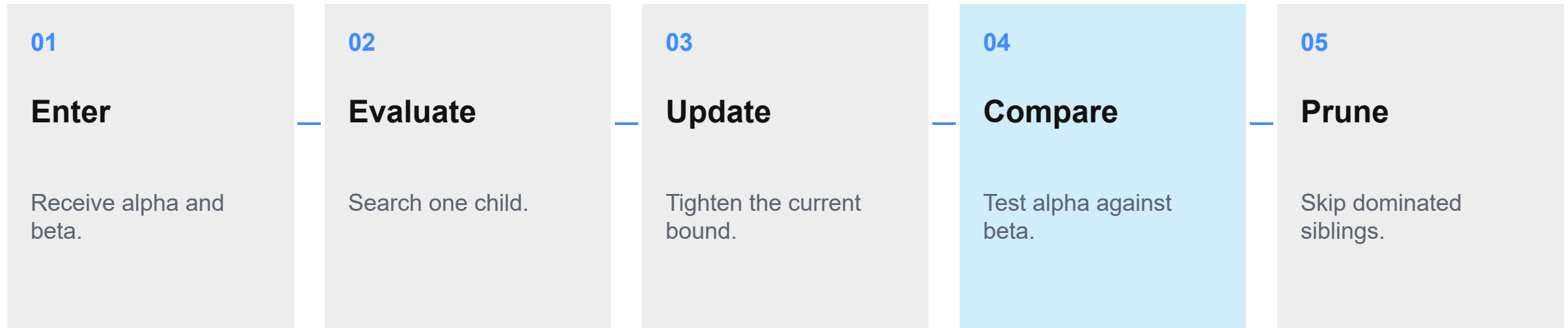
Plain minimax

- Visits every node to depth m .
- Returns exact minimax value.
- Order does not affect value.
- Time $O(b^m)$.

Alpha-beta

- Skips provably irrelevant nodes.
- Returns the same exact value.
- Order strongly affects work.
- Best case approaches $O(b^{(m/2)})$.

A cutoff follows a local proof



The comparison is valid because an ancestor already owns a dominating alternative.

Alpha-beta threads bounds through recursion

```
function MAX_VALUE(s, alpha, beta):  
    if cutoff(s): return evaluate(s)  
    v = -infinity  
    for a in ordered_actions(s):  
        v = max(v, MIN_VALUE(result(s,a), alpha, beta))  
        if v >= beta:  
            return v  
        alpha = max(alpha, v)  
    return v
```

MIN is symmetric

- Start $v = +\infty$.
- Cut when $v \leq \alpha$.
- Update $\beta = \min(\beta, v)$.
- Keep one utility perspective.

Worked alpha-beta cutoff

SCENARIO

MAX's first MIN child returns 5, so alpha becomes 5. A second MIN child first reveals value 3.

- The second child is controlled by MIN.
- Its current value is at most 3.
- MAX already has an alternative worth 5.
- Remaining siblings under that MIN node cannot improve MAX's choice.

Prune the remaining siblings; the root value remains 5.

Move ordering controls pruning efficiency

Best first

Strong bounds early

- MAX sees high values early.
- MIN sees low values early.
- Many cutoffs.
- Depth roughly doubles for same work.

Random

Moderate pruning

- Some useful early bounds.
- Variable node counts.
- Value remains exact.
- Seed for reproducibility.

Worst first

Weak bounds

- Cutoffs arrive late.
- Approaches plain minimax.
- Value remains exact.
- Ordering heuristic matters.

Checkpoint: identify the cutoff

CHECKPOINT

**At a MIN node with $\alpha=4$, the first child returns 2.
Can the remaining children be pruned?**

01

Update the MIN node's current value.

02

Compare v with α .

03

Explain the ancestor's existing alternative.

Expectimax models chance rather than hostility

MIN node

- Another agent chooses an action.
- Backup is minimum utility.
- Represents worst rational reply.
- Needs adversarial assumption.

Chance node

- Outcome is sampled from a distribution.
- Backup is probability-weighted mean.
- Represents stochastic dynamics.
- Needs calibrated probabilities.

Chance nodes back up expected value

$$V(s) = \sum_o P(o \mid s) V(\text{RESULT}(s,o))$$

- Outcomes o are mutually exclusive.
- Probabilities are nonnegative.
- Probabilities sum to 1.
- Child values use the same utility scale.

ENGINEERING IMPLICATION

Expectimax can prefer risk when a high payoff raises expected utility.

Minimax and expectimax can choose differently

SCENARIO

Safe action returns 4. Risky action returns 0 or 10 with probability 0.5 each.

- Minimax values Risky at $\min(0, 10) = 0$.
- Expectimax values Risky at $0.5 \cdot 0 + 0.5 \cdot 10 = 5$.
- Safe remains value 4.
- The preferred action depends on the model.

Minimax selects Safe; expectimax selects Risky under the stated probabilities.

Evaluation design remains a practical bottleneck

Features

State evidence

- Progress toward objective.
- Threat and opportunity.
- Mobility or control.
- Terminal conditions dominate.

Weights

Tradeoff scale

- Normalize magnitudes.
- Document sign and units.
- Avoid double counting.
- Validate on tactical cases.

Ordering

Search efficiency

- Reuse evaluation cheaply.
- Try promising moves first.
- Cache repeated states.
- Do not change backed-up rule.

Checkpoint: compute a chance backup

CHECKPOINT

A chance node reaches values -2, 4, and 10 with probabilities 0.2, 0.5, and 0.3. What value backs up?

01

Check that probabilities sum to 1.

02

Multiply each value by its probability.

03

Add the three contributions.

Cross-check exact optimization against the baseline

Every alpha-beta implementation should match plain minimax on small trees while expanding no more nodes.

Equivalence

Value and root action are correctness evidence; node count is optimization evidence.

- Generate small deterministic trees with known leaves.
- Compare values across multiple action orders.
- Test expectimax separately with normalized probability tables.

Key takeaways

- 01 — **Alpha-beta is an exact minimax optimization.**
- 02 — Alpha and beta are path-relative decision bounds.
- 03 — Move ordering changes work but not value.
- 04 — Expectimax replaces adversarial minimum with a probability-weighted mean.

Next: Week 4 generalizes sequential decision making under uncertainty with Markov decision processes.

Check your understanding

Q1

Trace alpha-beta on a two-ply tree.

Q2

Explain why a cutoff preserves value.

Q3

Derive the best-case square-root branching effect.

Q4

Compute a chance-node expectation.

Q5

Choose minimax or expectimax for a stated domain.